Policy Registry sandbox review

This package reviews `GravitasPolicyRegistry` at source commit `692022e1ca34a280f90a198d2455ac3535b3bce2`. It uses dummy accounts and tokens. It does not demonstrate a real sukuk issuance or an enforced sukuk transfer.

Read in this order

1. Sandbox access: install the tools and run the local chain.
2. Scenario 1: start with a simulated tokenised sukuk transfer. It includes the three answers about permission, bypass and outages.
3. Scenarios 2–6: pair approval, rejection, version changes, pause and unauthorised policy writes.
4. API specification: the two existing mock HTTP endpoints and the real registry calls through a blockchain node. There is no registry REST adapter in the repo.
5. Sample audit record and explanation: a composed local example, with the limits of the current audit trail.
6. Verification and gaps: what was run, what remains unverified and the owner-supplied tasks.

The combined `mb-sandbox-review-package.pdf` contains this starting page, access instructions, all six scenarios, the three answers and the sample audit record. The YAML and executable helpers remain in this folder.

Live today versus still missing

Area	Present and verified	Missing or still in development
Registry	Asset, router and executor lists; pause; version counter and change hashes. 86 existing Foundry tests passed.	Investor-specific checks: `subscriber` is ignored.
Testnet	Documented Arbitrum Sepolia registry has matching runtime bytecode; version 1; not paused.	No asset approved. Not a ready-to-run transfer demonstration.
HTTP API	Local `/v1/verify` and `/v1/policies` mock runs from a JSON file.	Registry-backed HTTP service and authentication are not implemented in the reviewed mock. Hosted access is unverified.
Transfer enforcement	Existing migration engines check assets inside their execution path.	No sukuk token/manager. The dummy token's direct and delegated transfers do not check the registry.
Audit	`PolicyUpdated(version, policyHash)` and returned subscription-check version.	No single contract record binding transfer, rule version and outcome.
Production	Repository documents testnet use.	Mainnet deployment, completed independent audit and external certification are not established by this review.

A whitelist is a list of addresses approved by the registry owner. It does not independently assess an instrument's terms. A passing check returns facts; it does not move tokens. A paused check reverts, meaning that the call fails. A real transfer integration must stop on that failure and must not provide another unchecked route.

Source references and every **TODO FOR ABDUSAMED**, T1–T7, are in the linked verification and access guides.

Local access and API scope

Run the six scenarios on a private local chain, not with real funds. Install Git, Python 3, Node.js 22 with npm, and Foundry 1.9.0. On Linux, macOS or Windows Subsystem for Linux, install Foundry with the official installer:

```
curl -L https://foundry.paradigm.xyz | bash
export PATH="$HOME/.foundry/bin:$PATH"
foundryup --install 1.9.0
```

Clone the review branch and test the actual source:

```
git clone --branch mb-sandbox-review --recurse-submodules \
  https://github.com/AbZe628/gravitas-protocol.git
cd gravitas-protocol
forge build
forge test
```

Expected: 86 tests pass, zero fail. Build warnings may appear. In terminal 1 start a fresh Anvil test blockchain. Do not expose its port:

```
anvil --host 127.0.0.1 --port 8545 --chain-id 31337 \
  --mnemonic 'test test test test test test test test test test test junk'
```

In terminal 2, from the repository root:

```
bash mb-sandbox-review/scripts/setup-local.sh
source /tmp/mb-sandbox-env.sh
python3 mb-sandbox-review/scripts/check-local.py
```

Expected: deployed registry and dummy-token addresses, policy version 3, six PASS lines, local evidence, then a restored version-3 baseline. The helper uses the repo's existing contracts and public Anvil accounts: owner account 0, executor account 1, recipient account 2. No private key or hosted API key is needed. The dummy token is GUSD, not a sukuk contract. The setup refuses a non-fresh chain.
For manual mutating scenarios, save a local snapshot first and restore it afterwards:

```
SNAP=$(cast rpc evm_snapshot --rpc-url "$RPC" | tr -d '"')
# Run one scenario's commands individually; some must fail.
cast rpc evm_revert "$SNAP" --rpc-url "$RPC"
```

Two different interfaces

The YAML specification covers the existing mock HTTP `POST /v1/verify` and `GET /v1/policies`, plus node JSON-RPC `eth_call` for registry checks and reads. The mock is file-driven and unauthenticated; it is not an HTTP wrapper for the registry. No hosted endpoint is verified. Its dependency audit includes a critical reported vulnerability, so keep it local. Do not use the mock as a transfer approval control.
Use the full access guide for the exact mock startup, dummy addresses, optional read-only testnet commands and all owner-supplied tasks. Use the OpenAPI file for encodings, responses and limitations. Links in this PDF point to the review branch. Local tests need neither hosted access nor testnet write access.

01 — Simulated tokenised sukuk transfer

Setup

Reviewed base: `692022e1ca34a280f90a198d2455ac3535b3bce2`, branch `mb-sandbox-review`.
Sukuk are asset-linked certificates; none are implemented here. `contracts/MockERC20.sol` creates **Gravitas USD / GUSD**, a dummy ERC-20 (standard balance-and-transfer token), not a sukuk. All actors and addresses below are dummy local accounts.
Use `../sandbox-access.md` for setup, not these steps to deploy. Baseline: `RPC=http://127.0.0.1:8545`; freshly deployed `REGISTRY`; existing MockERC20 `ASSET`; `OWNER` = Anvil0, `EXECUTOR` = Anvil1, `RECIPIENT` = Anvil2, `UNKNOWN=0x0000000000000000000000000000000000000999`. OWNER must hold at least one GUSD. Constructor grants OWNER executor status at version 1; asset whitelist at version 2; EXECUTOR whitelist at version 3. A whitelist is an explicit allowed-address list. An executor is the caller authorized to request the combined check.

1. What is permitted or prohibited?

The registry stores allowed assets and callers. These calls are read-only: they do not move tokens. Run separately; expected rejections stop only that call. A revert means the contract rejects the call.

```
# Permitted: returns 3
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$EXECUTOR" --rpc-url "$RPC"
# Prohibited asset: GPR: Asset not Shariah-compliant
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$UNKNOWN" --from "$EXECUTOR" --rpc-url "$RPC"
# Prohibited caller: GPR: Calling contract not an authorized executor
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$UNKNOWN" --rpc-url "$RPC"
# Also returns 3: subscriber is ignored, not an eligibility check
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$UNKNOWN" \
  "$ASSET" --from "$EXECUTOR" --rpc-url "$RPC"
```

2. Which routes bypass the check?

Demonstrate a real local transfer while the asset is prohibited and the registry paused. `--unlocked` uses the local node's OWNER account; never use this on a remote node.

```
cast send "$REGISTRY" 'setAssetCompliance(address,bool)' "$ASSET" false --from "$OWNER" \
  --unlocked --rpc-url "$RPC"
# Now version 4; check rejects the asset
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$EXECUTOR" --rpc-url "$RPC"
cast send "$REGISTRY" 'pause()' --from "$OWNER" --unlocked --rpc-url "$RPC"
# Now rejects with EnforcedPause()
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$EXECUTOR" --rpc-url "$RPC"
# Record both balances before and after this transfer
cast call "$ASSET" 'balanceOf(address)(uint256)' "$OWNER" --rpc-url "$RPC"
cast call "$ASSET" 'balanceOf(address)(uint256)' "$RECIPIENT" --rpc-url "$RPC"
cast send "$ASSET" 'transfer(address,uint256)' "$RECIPIENT" 1000000000000000000 --from \
  "$OWNER" --unlocked --rpc-url "$RPC"
cast call "$ASSET" 'balanceOf(address)(uint256)' "$OWNER" --rpc-url "$RPC"
cast call "$ASSET" 'balanceOf(address)(uint256)' "$RECIPIENT" --rpc-url "$RPC"
```

Expected: OWNER loses one GUSD, RECIPIENT gains one, and `Transfer` is emitted. Registry remains paused at version 4. Inherited `transfer` and approved-spender `transferFrom` never consult it. TeleportV2's owner-only `rescueTokens` also has no registry check, but moves only tokens held by Teleport.

TeleportV2/V3 `onlyAuthorized` owner short circuit skips **executor authorization only**. Migration still calls `verifyAssetCompliance`; ownership cannot bypass prohibited assets or a paused registry. The owner-pause test below proves passage to deadline validation, not successful migration.

Closing the transfer gap requires a token or manager that calls the guarded registry during execution on every balance-moving route, including delegated transfers and any custody or administrative route. An optional pre-check is not enough. That sukuk integration is absent here; no contract was changed to add it. Use a local snapshot and restore it before running another scenario, or run the snapshot-restoring `scripts/check-local.py` helper from the package root.

3. What if the API or registry is unavailable?

API means application programming interface. The separate HTTP mock uses a local JSON policy file, not live registry calls; its log is not transfer enforcement. `verify-client.ts` exits on request failure and executes no transfers. An integrating application must stop on an unavailable decision, not treat it as permission. Failed registry calls cannot authorize a guarded migration; direct GUSD transfer remains independent. If the local RPC itself is down, these commands cannot submit transactions.

The guarded migration paths **fail closed**: failure blocks execution. This is decided by the registry's `whenNotPaused` and the migration contracts' required asset checks; neither migration catches a registry failure and continues. A direct GUSD transfer does not consult the API or registry at all, so their outage cannot block it. No production HTTP transfer client is present: fail-open versus fail-closed there is an integration choice that still needs implementation and testing. In `ComplianceService.gatedRead`, read failures are thrown, not converted to permission.

Existing tests, references, and limits

Run from repository root; the underlying tests passed in the full 86-test review run:

```
forge test --match-path test/GravitasSubscriptionIntegration.t.sol -vv
forge test --match-path test/foundry/PausableTests.t.sol --match-test \
  'test_Registry_(VerifyFunctionsRevertWhenPaused|PauseHaltsExecutorAuthorizationSystemWide|
OwnerNotLockedOutOfAuthGateByRegistryPause)' \
  -vv
forge test --match-path test/foundry/TeleportFlow.t.sol --match-test '^test_V2_RescueTokens' -vv
```

Subscription tests: `test_CheckSubscriptionCompliance_Success`, `test_CheckSubscriptionCompliance_RevertOnNonCompliantAsset`, `test_CheckSubscriptionCompliance_RevertOnUnauthorizedExecutor`.

Sources: `contracts/GravitasPolicyRegistry.sol` — `checkSubscriptionCompliance`, `_recordChange`, `pause`; `contracts/MockERC20.sol` — constructor; `lib/openzeppelin-contracts/contracts/token/ERC20/ERC20.sol` — `transfer`, `transferFrom`, `_update`; `contracts/TeleportV2.sol` — `onlyAuthorized`, `migrateLiquidityV2`, `rescueTokens`; `contracts/TeleportV3.sol` — `onlyAuthorized`, `executeAtomicMigration`; `integration-kit/mock-server/server.js` — `decide`, `/v1/verify`; `integration-kit/sdk-examples/verify-client.ts` — `run`; `integration-kit/README.md` — flow.

No existing dedicated MockERC20 bypass test was found. Separate checks and transfers have no atomic (all-in-one-transaction) rule-version audit in the existing token contract. No sukuk issuance, holder eligibility, or legal rights are demonstrated.

02 — Permitted pair check

Setup

Reviewed base: `692022e1ca34a280f90a198d2455ac3535b3bce2`, branch `mb-sandbox-review`.

Use the existing `GravitasSubscriptionIntegrationTest` fixture. A fixture is the setup created for each test. `setUp()` deploys a fresh, unpaused registry and whitelists dummy asset A (`0x0000000000000000000000000000000000000100`). A whitelist is an explicit list of allowed addresses. The constructor grants the owner executor status at policy version 1; asset

A is added at version 2; the fixture's executor is added at version 3. Executor status is not checked by either function in this scenario.
All addresses are dummy. These tests use address entries, not deployed token contracts or token balances.

Steps

1. In `test_VerifyAssetCompliance`, read `verifyAssetCompliance(A)` and read it for dummy unlisted address `0x000000000000000000000000000000000000dead`.
2. In `test_AreTokensCompliant`, the owner whitelists dummy asset B (`0x0000000000000000000000000000000000000101`). This adds policy version 4 in that test's fresh fixture.
3. Read `areTokensCompliant(A, B)`.
4. Read `areTokensCompliant(A, unlistedAddress)`.

Expected result

The single-asset reads return `true` for A and `false` for the unlisted address. The pair read returns `true` when both entries are whitelisted, and `false` when one entry is not.
The contract uses &&, meaning both conditions must be true. Both functions are read-only: they return a Boolean (`true` or `false`) and do not move tokens or update policy.

Existing proof commands

Run from the repository root:

```
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_VerifyAssetCompliance' -vv
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_AreTokensCompliant' -vv
```

Sources and limits

- `contracts/GravitasPolicyRegistry.sol`: `setAssetCompliance`, `verifyAssetCompliance`, `areTokensCompliant`.
- `test/GravitasSubscriptionIntegration.t.sol`: `setUp`, `test_VerifyAssetCompliance`, `test_AreTokensCompliant`.
- `docs/TECHNICAL_SPEC.md`, section 1, Invariant 2: documented both-assets rule; the contract and assertions define this scenario's evidence.

The pair test covers both allowed and the second entry unlisted. It does not separately assert first-entry-false or both-false cases; the contract expression implies those results. Neither function checks amount, recipient, investor identity, or caller authorization. An unlisted asset returns `false`; a paused registry instead reverts (ends the call with an error). No transfer, swap, settlement, or external approval is demonstrated. The named tests passed in the full review run. See ../verification.md for execution evidence.

03 — Prohibited asset or executor

Setup

Reviewed base: `692022e1ca34a280f90a198d2455ac3535b3bce2`, branch `mb-sandbox-review`.
The existing `GravitasSubscriptionIntegrationTest.setUp()` creates a fresh, unpaused registry. It grants the owner executor status in the constructor (version 1), whitelists a dummy asset (version 2), and authorizes a dummy executor (version 3). A whitelist is an explicit allowed-address list. An executor is a caller allowed to request the combined subscription check.
Optional local setup uses `../sandbox-access.md`: `RPC=http://127.0.0.1:8545`, fresh `REGISTRY`, existing MockERC20 `ASSET`, `OWNER=Anvil0`, `EXECUTOR=Anvil1`, `RECIPIENT=Anvil2`, and `UNKNOWN=0x0000000000000000000000000000000000000999`. Keep UNKNOWN off both lists and the registry unpaused; baseline version is 3.

Steps and expected result

1. Call the combined check from the authorized executor with an unapproved token. Expect a revert, `GPR: Asset not Shariah-compliant`, as asserted by `test_CheckSubscriptionCompliance_RevertOnNonCompliantAsset`.
2. Call it from an unauthorized address with the approved asset. Expect a revert, `GPR: Calling contract not an authorized executor`, as asserted by `test_CheckSubscriptionCompliance_RevertOnUnauthorizedExecutor`.

Optional local simulations:

```
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$UNKNOWN" --from "$EXECUTOR" --rpc-url "$RPC"
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$UNKNOWN" --rpc-url "$RPC"
```

`msg.sender` means the immediate caller: the account supplied by `--from` here, or a calling contract in an integration. `subscriber` is only an argument and is not checked. Supplying an authorized address as subscriber cannot authorize an unauthorized caller.

A Solidity `require` aborts the call when its condition fails. These failures do **not** return `false` or a policy version. The asset check comes first; if both checks fail, its error takes priority.

By contrast, `verifyAssetCompliance(unknown)` and `verifyExecutorStatus(unauthorized)` return `false` while unpaused. Those boolean reads do not themselves reject a subscription; the caller must enforce their result. When paused, both verification functions and the combined check revert with `EnforcedPause()`.

Existing Forge proof commands

Run from the repository root:

```
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_CheckSubscriptionCompliance_RevertOnNonCompliantAsset' -vv
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_CheckSubscriptionCompliance_RevertOnUnauthorizedExecutor' -vv
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_Verify(AssetCompliance|ExecutorStatus)' -vv
```

Sources and limits

Sources: `contracts/GravitasPolicyRegistry.sol`, `checkSubscriptionCompliance`, `verifyAssetCompliance`, `verifyExecutorStatus`; `test/GravitasSubscriptionIntegration.t.sol`, the named tests; `docs/KNOWN-ISSUES.md`, "One compiler warning" (unused subscriber).

The named tests passed in the full review run. These are read-only checks, not token transfers or completed subscriptions. The tests use dummy addresses, not a deployed subscription manager. Subscriber eligibility is unimplemented; dual-failure ordering is established by code, not a dedicated test. Registry flags are owner-set policy, not external approval.

04 — Rule changed to a new version

Setup

Review base: `692022e1ca34a280f90a198d2455ac3535b3bce2`, branch `mb-sandbox-review`. All accounts and assets are dummy.

A whitelist is an allowed-address list. An executor is an authorized caller of the combined subscription check. A policy version numbers a real list change, not every transaction.

Use a fresh baseline from sandbox-access.md: local RPC `http://127.0.0.1:8545`, `REGISTRY`, existing MockERC20 `ASSET`, `OWNER=Anvil0`, `EXECUTOR=Anvil1`, and `RECIPIENT=Anvil2`. Constructor executor grant to OWNER creates version **1**; asset addition creates **2**; EXECUTOR grant creates **3**. `GravitasSubscriptionIntegrationTest.setUp()` performs this same sequence. Each Forge test creates its own fixture, meaning its own fresh setup.

Steps

1. Check constructor version 1 with `test_GenesisIsTheDeployerGrant`.

2. Check chained changes with `test_EachVersionFoldsInTheOneBeforeIt` and `test_ARealChangeStillAdvancesTheRecord`. The first test adds an asset at 2, a **router**, not an executor, at 3, and revokes the asset at 4.
3. Check duplicate writes with `test_RewritingTheSameValueIsNotAPolicyChange`, `test_RewritingARouterChangesNothing`, `test_RewritingAnExecutorChangesNothing`, and `test_ARepeatedWriteEmitsNothing`.
4. Optional local exercise, inside the snapshot described in sandbox-access: repeat `setAssetCompliance(ASSET,true)` as OWNER; version stays 3. Set it false; version becomes 4. Set it true again; version becomes 5. Pause and unpause; version stays 5. Restore the snapshot afterward. Subscription reads must use `--from "$EXECUTOR"`.

Expected result

A hash is a fixed-size fingerprint. The hash chain includes the previous fingerprint in each new one. Exact Solidity formula for every real change:

```
H[0] = bytes32(0);
H[v] = keccak256(abi.encode(H[v-1], field, subject, status));
```

`field` is `keccak256("asset")`, `keccak256("router")`, or `keccak256("executor")`; `subject` is the address and `status` the new boolean. Use `abi.encode`, not `abi.encodePacked`. Duplicate writes emit no change event and keep both version and hash unchanged. Order matters: `test_OrderOfChangesIsPartOfTheCommitment` checks this.

Existing proof commands

Run from the repository root:

```
forge test --match-contract '^PolicyCommitment' -vv
forge test --match-contract '^GravitasSubscriptionIntegrationTest' --match-test \
  '^test_CheckSubscriptionCompliance_Success' -vv
forge test --match-contract '^PausableTests' --match-test '^test_Registry_PauseUnpause' -vv
```

Sources and limits

`contracts/GravitasPolicyRegistry.sol`: constructor, three setters, `_recordChange`, `pause`, `unpause`, `checkSubscriptionCompliance`. Tests: `test/foundry/PolicyCommitment.t.sol`, `test/GravitasSubscriptionIntegration.t.sol`, `test/foundry/PausableTests.t.sol`. Documentation: `docs/whitepaper.md` §3.3.
`policyHistory` maps versions to hashes, **not historical full states**. Reconstruction requires the ordered changes. `test_AnObserverCanReplayTheChainFromEvents` recomputes known changes; it does not decode recorded logs. Pause-version stability follows from code; the existing pause test does not assert it. The named tests passed in the full review run.

05 — Paused registry and unavailable service

Setup

Use the fresh local baseline in sandbox-access.md: `$RPC` is `http://127.0.0.1:8545`; `$REGISTRY`, `$ASSET`, `$OWNER`, `$EXECUTOR`, `$RECIPIENT` and `$UNKNOWN` are dummy local values. Version is 3: constructor executor grant, asset whitelist entry, then executor whitelist entry. A whitelist is an explicit allowed-address list.
A **revert** rejects a contract call and rolls back its changes. **Fail closed** means failure does not grant permission. A **raw getter** reads stored data without enforcing permission.

Steps and expected result

From the repository root, run these optional local probes individually; the five paused checks are expected to fail, so do not stop at the first error:

```
source /tmp/mb-sandbox-env.sh
read_registry() {
```

```
  cast call "$REGISTRY" "$@" --from "$EXECUTOR" --rpc-url "$RPC"
}
read_registry 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" "$ASSET"
SNAP=$(cast rpc evm_snapshot --rpc-url "$RPC" | tr -d '"')
cast send "$REGISTRY" 'pause()' --from "$OWNER" --unlocked --rpc-url "$RPC"
read_registry 'paused()(bool)'
read_registry 'verifyAssetCompliance(address)(bool)' "$ASSET"
read_registry 'areTokensCompliant(address,address)(bool)' "$ASSET" "$ASSET"
read_registry 'verifyRouterAuthorization(address)(bool)' "$UNKNOWN"
read_registry 'verifyExecutorStatus(address)(bool)' "$EXECUTOR"
read_registry 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" "$ASSET"
read_registry 'isAssetCompliant(address)(bool)' "$ASSET"
read_registry 'isRouterAuthorized(address)(bool)' "$UNKNOWN"
read_registry 'isExecutor(address)(bool)' "$EXECUTOR"
read_registry 'currentVersion()(uint256)'
read_registry 'getPolicyVersion()(uint256)'
cast send "$REGISTRY" 'unpause()' --from "$OWNER" --unlocked --rpc-url "$RPC"
read_registry 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" "$ASSET"
cast rpc evm_revert "$SNAP" --rpc-url "$RPC"
```

Before pause and after unpause, the subscription check returns 3. While paused, `paused()` returns `true`; all five enforcement checks revert with `EnforcedPause()`, not `false`. Raw mapping reads return `true`, `false`, `true`; both version reads return 3. Pause/unpause do not change policy version. Restore the snapshot even if a probe surprises you.
With the optional mock HTTP server stopped, run:

```
curl --max-time 2 http://127.0.0.1:8787/v1/policies
```

Expect a connection error, not an `ALLOW` decision. This is only a transport probe.

Existing Forge proof

```
forge test --match-path test/foundry/PausableTests.t.sol --match-contract '^PausableTests' \
  --match-test '^test_Registry_' -vv
```

This selects existing functions `test_Registry_PauseUnpause`, `test_Registry_OnlyOwnerCanPause`, `test_Registry_VerifyFunctionsRevertWhenPaused`, `test_Registry_PauseHaltsExecutorAuthorizationSystemWide`, `test_Registry_UnpauseRestoresVerification` and `test_Registry_OwnerNotLockedOutOfAuthGateByRegistryPause`.

Sources and limits

`contracts/GravitasPolicyRegistry.sol`: `pause`, `unpause`, five guarded checks, public mappings and `getPolicyVersion`. `contracts/TeleportV3.sol`: `onlyAuthorized` rejects non-owner executors through the paused registry; the owner bypasses that authorization check, but not the later asset check. `contracts/TeleportV2.sol`: `onlyAuthorized` and `migrateLiquidityV2` have guarded executor/asset paths, though its router read is raw. See `integration-kit/PROTOCOL-TESTING.md`, sections 4–5.
These Forge tests do not assert paused raw-read/version behavior; the probes cover that gap. Raw reads enforce nothing. `integration-kit/mock-server/server.js` reads a fixture, not the registry; `integration-kit/sdk-examples/verify-client.ts::run` exits on errors. No production HTTP outage handling or transfer executor is provided. Timeouts, retries and refusal to use stale permission need implementation and tests. The named tests passed in the full review run; scripts/check-local.py additionally ran the core local probes.

06 — Unauthorised rule change attempt

Setup

Use the dummy local baseline in sandbox-access.md, from the repository root. `$RPC` is `http://127.0.0.1:8545`. `$OWNER` is Anvil0, `$EXECUTOR` Anvil1, `$RECIPIENT` Anvil2; `$REGISTRY`, `$ASSET` and `$UNKNOWN` are dummy values. Version 3 records the constructor executor grant, asset whitelist entry and second executor grant.
`onlyOwner` allows only the current owner to call a function. An executor may pass execution checks but cannot change rules. `Ownable2Step` requires a proposed owner (`pendingOwner`) to accept before becoming owner. A **revert** rejects a call and rolls back its changes.

Steps and expected result

Optional local probes follow. Run individually: denied writes and the wrong acceptance must fail; continue after those expected errors. A snapshot saves local chain state for restoration.

```
source /tmp/mb-sandbox-env.sh
SNAP=$(cast rpc evm_snapshot --rpc-url "$RPC" | tr -d '"')
read_registry() {
  cast call "$REGISTRY" "$@" --from "$EXECUTOR" --rpc-url "$RPC"
}
write_registry() {
  local caller="$1"; shift
  cast send "$REGISTRY" "$@" --from "$caller" --unlocked --rpc-url "$RPC"
}
read_registry 'policyHistory(uint256)(bytes32)' 3
for fn in setAssetCompliance setRouterAuthorization setExecutorStatus; do
  write_registry "$EXECUTOR" "$fn(address,bool)" "$UNKNOWN" true
done
write_registry "$OWNER" 'transferOwnership(address)' "$RECIPIENT"
read_registry 'owner()(address)'
read_registry 'pendingOwner()(address)'
write_registry "$RECIPIENT" 'setAssetCompliance(address,bool)' "$UNKNOWN" true
write_registry "$EXECUTOR" 'acceptOwnership()'
read_registry 'isAssetCompliant(address)(bool)' "$UNKNOWN"
read_registry 'currentVersion()(uint256)'
read_registry 'policyHistory(uint256)(bytes32)' 3
write_registry "$RECIPIENT" 'acceptOwnership()'
read_registry 'owner()(address)'
read_registry 'pendingOwner()(address)'
cast rpc evm_revert "$SNAP" --rpc-url "$RPC"
```

All denied operations revert with `OwnableUnauthorizedAccount(caller)`. Before correct acceptance, owner remains `$OWNER`, pending owner is `$RECIPIENT`, the unknown asset remains `false`, version stays `3` and its hash stays unchanged. Correct acceptance changes owner to `$RECIPIENT` and clears pending owner to zero. Restore the snapshot even after unexpected results.

Existing Forge proof

```
forge test --match-path test/foundry/PolicyCommitment.t.sol --match-contract \
  '^PolicyCommitment' --match-test \
  '^test_(OnlyTheOwnerWrites|OnlyTheOwnerWritesToEveryRegister)' -vv
forge test --match-path test/GravitasPolicyRegistryMultisig.t.sol --match-contract \
  '^GravitasPolicyRegistryMultisigTest' --match-test \
```

```
'^test_Ownable2Step_(TransferRequiresAcceptance|AttackerCannotAccept|OnlyOwnerCanPropose)' \
-vv
```

Selected tests: `test_OnlyTheOwnerWrites`, `test_OnlyTheOwnerWritesToEveryRegister`, `test_Ownable2Step_TransferRequiresAcceptance`, `test_Ownable2Step_AttackerCannotAccept` and `test_Ownable2Step_OnlyOwnerCanPropose`. The rejection tests expect any revert, not the exact error type.

Sources and limits

`contracts/GravitasPolicyRegistry.sol`: `setAssetCompliance`, `setRouterAuthorization`, `setExecutorStatus`, `_recordChange`. `lib/openzeppelin-contracts/contracts/access/Ownable.sol`: `onlyOwner`, `_checkOwner`; adjacent `Ownable2Step.sol`: `transferOwnership`, `pendingOwner`, `acceptOwnership`.
The five tests do not directly attempt a pending-owner write; that conclusion follows from the inherited owner check, with an optional probe above. Despite its filename, the multisig test uses dummy addresses, not a multiple-signature wallet. No real multisig or timelock (delayed execution controller) is established; see `docs/KNOWN-ISSUES.md`, "Ownership has not moved to the timelock." These controls cannot stop an authorised owner changing rules immediately. The named tests passed in the full review run; scripts/check-local.py additionally ran the core local probes.

Sample audit record: what it proves

`sample-audit-record.json` is a dummy local example assembled by the review helper. Its outer keys group actual contract arguments, return values, event fields and node responses. Those grouping keys are not a new registry event or stored audit schema. The file is **not** proof that a protected sukuk transfer or an approved external ruling exists.

Read the example

The transfer is 100 dummy `GUSD` units from Anvil account 0 to account 2. `Transfer.from`, `Transfer.to` and `Transfer.value` match the ERC-20 event in `lib/openzeppelin-contracts/contracts/token/ERC20/IERC20.sol`. `value` is `100000000000000000000` because this dummy token has 18 decimal places. Its contract is `contracts/MockERC20.sol`; no sukuk features are present.
The checked rule is the existing `checkSubscriptionCompliance(subscriber, subscriptionToken)` function in `contracts/GravitasPolicyRegistry.sol`. It requires the asset flag to be true and `isExecutor[msg.sender]` to be true. The JSON gives the exact call input, simulated caller and block tag. A successful raw `eth_call.response.result` ending in `03` decodes to `policyVersion = 3`. The check uses version 3's current lists. It does **not** check the investor, recipient, amount, instrument documents or an external certification.
`PolicyUpdated.version` and `PolicyUpdated.policyHash` use the actual event's field names. The helper reads the hash from `policyHistory(3)`; the same version/hash pair was emitted when the second executor was granted. Version 1 is the constructor grant, version 2 approves the asset, and version 3 grants this executor. `_recordChange` hashes the previous hash, register identifier, changed address and boolean value. This is an ordered change commitment, not the text or identifier of a board-approved ruling. The registry stores no such ruling document.
`eth_getTransactionReceipt.transactionHash` is the separate token transfer's transaction reference. `status = 0x1` means that transfer succeeded, **not** that it applied a registry rule. `logs` contains its real `Transfer` event shape. `eth_getBlockByNumber.timestamp` is the transfer block's timestamp in hexadecimal seconds since 1 January 1970; the registry event itself contains no timestamp. Block and receipt fields come from the blockchain node, not registry storage.

The gap

The check was simulated at block `0x4`; the transfer was a different operation at block `0x5`. The check emitted no event and has no transaction hash. The token transfer did not call the registry. Therefore this file proves only **a successful version-3 check followed by a separate dummy transfer**. It cannot honestly prove "version 3 was enforced by this transfer". The version could change, or the registry could pause, between a pre-check and execution.
Neither `LiquidityMigrated` in `TeleportV2.sol` nor `LiquidityTeleported` in `TeleportV3.sol` emits the registry version. The HTTP mock's audit line contains `ts`, `requestId`, `channel`, `asset`, `amount`, `decision` and `reason`, but no on-chain version, policy hash or transaction reference.

TODO FOR ABDUSAMED T6: an actual transfer integration must perform the check and record its returned version during the same transaction. Retain the relevant policy-change events and map the version to the governing approval outside this registry. That integration is not in this repo. No contract code was changed to manufacture it.
Reproduce the external fixture with `python3 mb-sandbox-review/scripts/check-local.py` after local setup. Local transaction hashes and timestamps can differ on a new run. All values are sandbox-only; no real money or customer data is used.

Sample audit JSON

This is the complete dummy local fixture. The previous section explains its sources and limits. It is not a single contract audit event.


```
{
  "eth_chainId": "0x7a69",
  "eth_call": {
    "request": {
      "jsonrpc": "2.0",
      "id": 1,
      "method": "eth_call",
      "params": [
        {"from": "0x70997970C51812dc3A010C7d01b50e0d17dc79C8", "to":
"0x5FbDB2315678afecb367f032d93F642f64180aa3", "data":
"0xdc65a7b500000000000000000000000003c44cdddb6a900fa2b585dd29
9e03d12fa4293bc000000000000000000000000e7f1725e7734ce288f8367e1bb143e90bb3f0512"},
        "0x4"
      ]
    },
    "response": {"jsonrpc": "2.0", "id": 1, "result":
"0x0000000000000000000000000000000000000000000000000000000000000003"}
  },
  "checkSubscriptionCompliance": {"subscriber": "0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC",
"subscriptionToken": "0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512", "policyVersion": "3"},
  "PolicyUpdated": {"version": "3", "policyHash":
"0x9ed40a743048349cd7d7373b555a582ef208f2b91788b460fa34c66e7fa4fc53"},
  "Transfer": {"from": "0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266", "to":
"0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC", "value": "10000000000000000000"},
  "eth_getTransactionReceipt": {
    "transactionHash": "0x412dd06a728b5cf6c46e4e18cc5d9130136e55610c07a16ae1494b0bbcf6af76",
    "blockNumber": "0x5",
    "status": "0x1",
    "from": "0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266",
    "to": "0xe7f1725e7734ce288f8367e1bb143e90bb3f0512",
    "logs": [
      {
        "address": "0xe7f1725e7734ce288f8367e1bb143e90bb3f0512",
        "topics": [
          "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
          "0x000000000000000000000000f39fd6e51aad88f6f4ce6ab8827279cfffb92266",
          "0x0000000000000000000000003c44cdddb6a900fa2b585dd299e03d12fa4293bc"
        ],
        "data": "0x0000000000000000000000000000000000000000000000056bc75e2d63100000",
```

```
        "blockHash": "0x645fc508e40b688c32c99d4f5bbd851134b566de5e6114f1183886f604fd3606",
        "blockNumber": "0x5",
        "blockTimestamp": "0x6af1d6cb",
        "transactionHash": "0x412dd06a728b5cf6c46e4e18cc5d9130136e55610c07a16ae1494b0bbcf6af76",
        "transactionIndex": "0x0",
        "logIndex": "0x0",
        "removed": false
      }
    ]
  },
  "eth_getBlockByNumber": {"hash": "0x645fc508e40b688c32c99d4f5bbd851134b566de5e6114f1183886f60
4fd3606", "number": "0x5", "timestamp": "0x6af1d6cb"}
}
```


Verification and owner-supplied tasks

Foundry 1.9.0: 86 existing tests passed, zero failed or skipped. Fourteen scenario-specific test commands also selected passing tests. Six local probe groups passed, including direct transfer, delegated-transfer bypass, paused checks and pending-owner refusal.
The documented Arbitrum Sepolia registry runtime matched the reviewed source exactly (3244 bytes, metadata included). At the dated read on 9 November 2026 it was version 1 and not paused, with no asset approval recorded. No testnet transaction was sent.
The remaining gaps are an absent enforced sukuk integration, ignored investor eligibility, a mock that trusts identifiers and does not read the registry, missing production HTTP outage handling, and no atomic transfer-to-version audit. Raw getters do not enforce pause. Engine owners skip executor authorization, not asset checks. V2 checks the router list; V3 uses fixed constructor venues without a router-list check. The V2 owner-only rescue route is separate and not compliance-checked.
Hosted access, production integration and external approval/certification were not verified. The mock dependency audit reported one critical, one high and one moderate vulnerability; this is not an exploitability assessment. This is a focused review, not a security audit.

Every TODO FOR ABDUSAMED

The complete operational instructions and wording are in sandbox-access.md. The seven owner-supplied tasks are:

- **T1:** arrange the reviewers' tool-equipped computer or an engineer to run the local review.
- **T2:** supply and test remote HTTP endpoint, authentication and credentials if hosted access is wanted.
- **T3:** supply verified testnet deployment/RPC, funded non-public wallets and authorised write access if a shared exercise is wanted.
- **T4:** supply/build the actual dummy sukuk transfer integration and guard every transfer route.
- **T5:** implement/test fail-closed HTTP outage handling, timeouts, retries and stale-decision policy.
- **T6:** bind the transfer and returned rule version in one transaction; retain policy-change evidence and map it to the governing approval.
- **T7:** harden the mock before hosting: dependency review, authentication and identifier/amount validation.

T2–T7 are not prerequisites for the local source review. They are prerequisites for claiming the corresponding remote, enforced-transfer or audit capability.
Full verification and source references.